the temperature of the environment the Pico board is being kept in. Let's run the code given below, taken from official Pico-MicroPython examples, to see the temperature sensor in action:

```
# Temperature Sensor reading

import machine
import utime

sensor_temp = machine.ADC(4)
conversion_factor = 3.3 / (65535)

while True:
    reading = sensor_temp.read_u16() * conversion_factor

# The temperature sensor measures the Vbe voltage of a biased
bipolar diode, connected to the fifth ADC channel
# Typically, Vbe = 0.706V at 27 degrees C, with a slope of
-1.721mV (0.001721) per degree.
    temperature = 27 - (reading - 0.706)/0.001721
    print(temperature)
    utime.sleep(2)
```

For a quick verification, just put your finger on the microcontroller chip (shown in Figure 1) and you'll see the temperature reported increasing a bit. Removing your finger from the chip settles the temperature to lower values again. Figure 3 shows temperature readings as read in the Thonny IDE shell on my end.

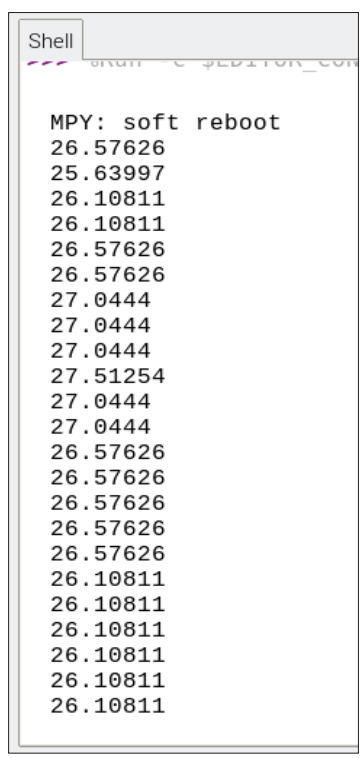

Figure 3: Values from the on-chip temperature sensor

Last but not the least, you can program the Pico board to run your code every time when powered up. Once you finalise your MicroPython code after testing, just hit the *Save* button in your Thonny IDE and save your code as *main.py* by clicking the Raspberry Pi Pico button in the *Where to save to?* dialogue box, as shown in the screenshot in Figure 4.

Now onwards, your faithful RPI Pico board will start

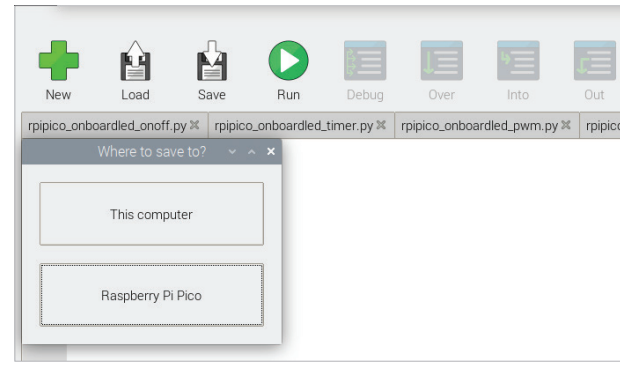

Figure 4: Saving the final code to the Pico board

running the MicroPython code provided in *main.py* whenever it's powered up. Isn't that great?

Real-world use of the PI Pico board

The RPI Pico has multiple GPIO pins covering various kinds of hardware interfaces as shown in Figure 5. You can experiment with these multiple GPIO pins to create more interesting functionalities using some external electronic components. You need an RPI Pico board with header pins, a breadboard, some resistors, some LEDs, and some jumper wires for connections. You'll be able to buy the additional components online or from a local electronics shop roughly within Rs 100.

The code shown below is extending the single onboard LED flashing example to multiple external LEDs. Just wire the LEDs and resistors with jumper wires on a breadboard, run the code and you should see your Pico board driving external LEDs in the form of three LEDs in a traffic light pattern. Figure 6 shows the wired circuit of the traffic light external LEDs. The logic of the code is simple: configure three GPIO pins (choose from GPIO pin numbers shown in Figure 5) for driving the different colours of external LEDs, ensuring only one colour LED is 'on' at a time with a delay in between.

```
# external LEDs Traffic Light

from machine import Pin
from time import sleep

rled = Pin(18, Pin.OUT)
gled = Pin(17, Pin.OUT)
yled = Pin(16, Pin.OUT)

def stoponred():
  global rled, gled, yled
  rled.on()
  gled.off()
  yled.off()
```